

strapdown-rs: A Simple Strapdown INS Implementation in Rust

James Brodovsky ^{1*}

¹ Temple University, United States  Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Kyle Niemeyer](#) 

Submitted: 05 June 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Inertial navigation systems (INSs) are critical for many applications in robotics, aerospace, and autonomous systems. They provide real-time estimates of position, velocity, and orientation using data from an inertial measurement unit (IMU) as well as other aiding sensors like GPS. Strapdown implementations of INSs are becoming increasingly common due to the proliferation of modern IMUs that do not require gyroscopic stabilization such as micro-electromagnetic systems (MEMS), fiber optic gyroscopes (FOGs), and ring laser gyroscopes (RLGs). These IMUs are popular for their lower size, weight, and power (SWaP) characteristics compared to historical spinning-mass gyroscopes and pendulous accelerometers, making them suitable for a wide range of applications including drones, robotics, and mobile devices. FOGs and RLGs in particular are becoming increasingly popular in the aerospace, maritime, and defense sectors due to their improved performance and lower cost. That said, such INSs are still highly reliant on GNSS position fixes for long-term accuracy, as even high-quality IMUs will drift over time without external corrections.

strapdown-rs is a Rust-based software library and simulation toolkit for implementing and testing strapdown inertial navigation systems (INS). It provides core functionality for processing inertial measurement unit (IMU) data to estimate position, velocity, and orientation using a strapdown mechanization model that is typical of modern systems, particularly in the low SWaP domain (cell phones, drones, robotics, UAVs, UUVs, etc.). Additionally, it provides a comprehensive simulation framework for safely testing INS performance under degraded and denied GNSS conditions—scenarios increasingly relevant for autonomous systems operating in contested or signal-denied environments.

strapdown-rs prioritizes correctness, numerical stability, and performance. It is built with extensibility in mind, allowing researchers and engineers to implement additional filtering, sensor fusion, or aiding algorithms on top of the base INS framework. This library is not intended to be a full-featured INS solution. Notably it does not have code for processing raw IMU or GPS signals from hardware and currently only implements a loosely-coupled INS.

The toolbox is designed for research, teaching, and development purposes and aims to serve the broader robotics, aerospace, and autonomous systems communities. The intent is to provide a high-performance, memory-safe, and modern cross-platform implementation of strapdown INS algorithms that can be easily integrated into existing systems or research workflows. The simulation framework enables users to generate synthetic navigation scenarios with controllable GNSS degradation, allowing for systematic testing and verification of INS algorithms under various failure modes.

Statement of Need

Existing inertial navigation system (INS) implementations are often fragmented across proprietary MATLAB code, legacy C/C++ systems, and Python prototypes—each with significant drawbacks for open, reproducible, and high-performance research. MATLAB

and Python are widely used for prototyping but are unsuitable for production or real-time systems, while C/C++ implementations are difficult to maintain and extend, lack modern tooling, and are rarely open source. This landscape forces researchers to translate algorithms between languages or pay for expensive commercial solutions, increasing complexity and the risk of errors, and makes it challenging to build reusable, extensible, and high-performance INS libraries. The motivating example for the development of this library is the notion of "vectorized" calculations.

Research in navigation algorithms, particularly alternative Positioning, Navigation, and Timing (PNT) methods, often necessitates the evaluation of other filter architectures such as particle filters or factor graph methods. Implementing these algorithms in an object-oriented manner in Python or MATLAB is straightforward, but the performance constraints make it difficult to evaluate them under realistic conditions. A common workaround is to implement the mathematics in a vectorized manner using NumPy or MATLAB's array operations. This can provide significant performance improvements, but it also introduces complexity, duplicates functionality, and can make the code less transparent and harder to debug. Additionally, vectorized implementations may not be suitable for real-time applications. There is a clear need for a modern, open, and high-performance toolkit that bridges the gap between rapid prototyping and compiled performance with a production environment in mind, enabling both research and production use without sacrificing performance, safety, or reproducibility. strapdown-rs addresses this gap by providing an INS library and simulation framework that is both accessible and suitable for high-performance applications.

The second need motivating the development of this library is to produce reliable and reproducible simulation of GNSS-denied scenarios. Such scenarios are a critical need for the navigation community, as collecting real-world data under signal denial typically requires expensive and logistically challenging field tests with specialized jamming hardware. Such tests are not only costly and difficult to repeat, but also raise regulatory and safety concerns, limiting their accessibility for most researchers. A robust simulation framework enables systematic evaluation of navigation algorithms under a wide range of degraded or denied GNSS conditions, supporting fair comparison, rapid iteration, and transparent reporting of results. By providing configurable, open-source tools for simulating GNSS outages, degradation, and spoofing, strapdown-rs empowers researchers to develop and validate robust navigation solutions without the barriers of hardware-based field-testing.

State of the Field

Several open-source tools exist for inertial navigation research, but each carries significant limitations for the use cases addressed by strapdown-rs. The gnss-ins-sim Python package (Fang, 2019) offers a simulation environment that produces similar outputs but is limited to Python's performance constraints and does not support GNSS degradation fault modeling. GTSAM (Dellaert, 2012) provides a powerful factor-graph framework in C++ and Python, but is a more general-purpose computer vision library rather than a dedicated INS toolkit, requiring substantial configuration overhead for straightforward strapdown navigation tasks. OpenVINS (Geneva et al., 2020) is a high-quality visual-inertial odometry system, but is tightly coupled to ROS and oriented toward camera-IMU fusion rather than GNSS-aided INS research. Numerous MATLAB implementations exist in academic repositories, but these are proprietary, non-portable, and unsuitable for deployment in performance-sensitive applications.

No existing open-source tool in a single package provides: a standalone library for strapdown mechanization suitable for production, a configurable GNSS degradation simulation framework with reproducible, seeded scenarios, and a configurable INS implementation. Contributing these capabilities to an existing project was not viable because the language choice, which is central to the safety and performance goals of this work, is incompatible with Python- or C++-based frameworks. strapdown-rs fills this gap as a self-contained, high-performance, reproducible toolkit aimed specifically at GNSS-denied navigation research.

94 Overview of Functionality

95 strapdown-rs provides both core navigation algorithms and simulation capabilities through
96 two main components: the strapdown-core library and the strapdown-sim binary. The
97 library contains four primary modules: earth, strapdown, filter, and sim (for simulation
98 utilities). The binary provides a command-line interface for running navigation simulations
99 with configurable GNSS degradation scenarios.

100 Core Library Modules

101 The earth module contains constants and functions related to the Earth's shape and other
102 geophysical features (gravity and magnetic field). The Earth is modeled as an ellipsoid with a
103 semi-major axis and a semi-minor axis ([National Geospatial-Intelligence Agency, 2014](#)). The
104 Earth's gravity is modeled as a function of the latitude and altitude using the Somigliana
105 method. The Earth's magnetic field is modeled using a dipole model ([NOAA NCEI Geomagnetic
Modeling Team and British Geological Survey, 2024](#)). This module relies on the nav-types
106 crate ([Nordmoen & Feuerstein, 2022](#)) for the coordinate types and conversions, but provides
107 additional functionality for calculating rotations for the strapdown navigation filters. This
108 permits the transformation of additional quantities (velocity, acceleration, etc.) between the
109 Earth-centered Earth-fixed (ECEF) frame and the local-level frame.
110

111 The strapdown module provides the forward mechanization equations for strapdown inertial
112 navigation systems. It provides a set of structs for modeling both IMU data and the base nine-
113 element strapdown state (latitude, longitude, and altitude; velocities north, east, and down;
114 and attitude). It includes an implementation for the local-level frame forward mechanization,
115 which is a common approach for strapdown INS and follows the equations from Chapter 5.4
116 of ([Groves, n.d.](#)). This module serves as the foundation for both dead reckoning and filtered
117 navigation solutions.

118 The filter module contains the core functionality for implementing strapdown INS algorithms,
119 primarily a loosely-coupled integration architecture according to Chapter 14.1.2 of ([Groves,
n.d.](#)). This module contains implementations of various inertial navigation filters, including an
120 Unscented Kalman Filter (UKF) and particle filter. These filters are used to estimate the state
121 of a strapdown inertial navigation system based on IMU measurements and other sensor data.
122 The filters use the strapdown equations (provided by the strapdown module) to propagate the
123 state in the local level frame. The module also provides measurement models for GPS position
124 and velocity updates.
125

126 The sim module provides utilities for simulation and testing, including data structures for
127 test data records (compatible with the Sensor Logger app format), navigation results, and
128 simulation functions for both dead reckoning and closed-loop navigation. Critically, this module
129 includes the event stream framework that enables simulation of GNSS degradation scenarios.

130 The messages module implements an event-driven architecture for simulating GNSS degradation.
131 It allows users to convert data contained in a tabular format (e.g. CSV files) into a sequence of
132 timestamped events that can be processed by the navigation filters, mimicking real-time sensor
133 data streams and allowing for reproducible testing of degraded input. This module provides:

- 134 ■ **GNSS Schedulers:** Control measurement availability through passthrough (all
135 measurements), fixed-interval sampling, or duty-cycle patterns (on/off periods)
- 136 ■ **Fault Models:** Simulate measurement corruption through various mechanisms including
137 degraded measurements (AR(1) noise processes), slow bias drift (position/velocity drift
138 with rotation), and hijacking/spoofing (position offset injection)
- 139 ■ **Event Streams:** Transform raw sensor data into sequences of IMU prediction steps
140 and measurement updates, with fault injection applied according to the configured
141 degradation model

142 Simulation Binary

143 The strapdown-sim binary provides a command-line interface for running navigation simulations
144 and generating synthetic datasets. It can be operated in three modes. In synthetic data
145 generation mode it can generate realistic trajectories from perfect kinematics based on an IMU
146 classification. In open-loop mode (dead reckoning) the system propagates the state using the
147 strapdown equations and IMU measurements without any corrections from aiding sensors. This
148 mode is only recommended when analyzing high-quality IMUs, as MEMS-grade sensors will
149 accumulate significant drift within seconds to minutes. In closed-loop mode (GNSS-aided INS)
150 the system uses a 15-state UKF (9 navigation states + 6 IMU bias states) to estimate the state
151 and correct it using GPS measurements. This mode supports extensive GNSS degradation
152 simulation capabilities that can be used to evaluate the robustness of navigation algorithms
153 under various failure modes such as:

- 154 ■ **Signal dropouts:** Simulated via duty-cycle schedulers with configurable on/off periods
- 155 ■ **Reduced update rates:** Implemented through fixed-interval schedulers
- 156 ■ **Measurement degradation:** AR(1) colored noise processes applied to position and velocity
157 measurements with configurable correlation and noise levels
- 158 ■ **Slow bias/drift:** Gradual position and velocity bias accumulation, optionally with rotation
159 to simulate realistic drift patterns
- 160 ■ **Spoofing/hijacking:** Position offset injection over specified time windows to simulate
161 spoofing attacks

162 These degradation scenarios can be configured individually or combined, and can be specified
163 either through command-line arguments or configuration files (JSON, YAML, or TOML). This
164 enables systematic testing of INS performance under controlled degradation conditions—critical
165 for developing robust navigation systems for contested environments.

166 Software Design

167 The central design decision in strapdown-rs is the choice of Rust as the implementation
168 language. Rust provides memory safety guarantees without garbage collection, eliminating
169 entire classes of bugs—buffer overflows, use-after-free, and data races—that are common in
170 navigation C/C++ codebases and difficult to detect in safety-critical systems. This comes at
171 no runtime performance cost relative to C/C++, making strapdown-rs suitable for future
172 real-time deployment, unlike Python-based alternatives.

173 The workspace is split into two crates: strapdown-core (library) and strapdown-sim
174 (CLI binary). This separation ensures that researchers who want only the navigation
175 algorithms—without the simulation scaffolding—can depend on the core library independently.
176 Within strapdown-core, trait-based abstractions (NavigationFilter, MeasurementModel)
177 decouple the filter implementations from specific sensor models, allowing new filters or
178 measurement sources to be added without modifying existing code.

179 The UKF was chosen over the more common Extended Kalman Filter (EKF) because the local-
180 level frame strapdown equations are nonlinear, and the unscented transform avoids first-order
181 linearization errors without requiring Jacobian derivations. The 9-state core strapdown state is
182 extended to 15 states in the UKF to estimate IMU accelerometer and gyroscope biases, which
183 are the dominant error sources for MEMS-grade sensors. The event-driven GNSS simulation
184 architecture deliberately separates measurement scheduling, fault injection, and filter update
185 logic into independent components, enabling systematic scenario construction and reproducible
186 experiments.

187 Key Technical Contributions

188 The primary technical contributions of strapdown-rs to the open-source navigation community
189 are:

1. A well-documented, numerically stable implementation of the local-level frame strapdown equations following (Groves, n.d.), providing a reusable foundation for navigation algorithm development.
2. A reference implementation of a 15-state UKF-based INS with GPS position and velocity aiding, suitable for MEMS-grade IMUs commonly found in robotics and low-SWaP applications.
3. A comprehensive toolkit for safely simulating INS performance under various GNSS degradation scenarios without requiring real-world testing in denied environments. The event-driven architecture separates concerns between measurement scheduling, fault injection, and navigation filtering, enabling researchers to systematically evaluate algorithm robustness.

These contributions make strapdown-rs particularly valuable for researchers developing alternative or complementary navigation algorithms for autonomous systems that must operate in contested or signal-denied environments, such as urban canyons, indoor spaces, or GPS-jammed areas.

Research Impact Statement

strapdown-rs has been developed in direct support of ongoing navigation research at Temple University. The particle filter implementation and GNSS degradation simulation framework have been used in research exploring geophysical anomaly based navigation in a UKF (Brodovsky & Dames, 2026a) and Rao-Blackwellized particle filter (Brodovsky & Dames, 2026b) approaches to geophysical anomaly-aided navigation, with associated data processing pipelines and analysis built on top of the strapdown-core library. This work has produced processed datasets and reproducible experimental configurations that are archived alongside the software, providing the community-readiness signals that support external replication.

The software's design for reproducibility—seeded random number generation, configuration-file-driven scenarios, and deterministic strapdown propagation—directly addresses a gap identified by the navigation research community: the lack of a publicly available, language-agnostic, reproducible baseline for evaluating GNSS-denied INS algorithms. Researchers can clone the repository, obtain the pre-processed input datasets, and reproduce published simulation results without access to proprietary tools or hardware.

As a Cargo-published crate, strapdown-core is available for direct dependency by other Rust projects. The clean trait-based interface for filters and measurement models is designed to lower the barrier for researchers wishing to implement and compare novel filtering algorithms—such as factor graph methods or learned observation models—against the provided UKF baseline, using identical strapdown mechanization and GNSS degradation scenarios.

AI Usage Disclosure

Generative AI tools, specifically Claude (Anthropic), were used during the development of this software and paper. AI was used in a pair-programming capacity during implementation of several modules in strapdown-core and strapdown-sim, and as a copy editor for documentation strings and this paper. All AI-generated or AI-assisted code was reviewed by the author, validated against the reference equations in (Groves, n.d.), and verified through the project's unit and integration test suite (cargo test --workspace). All AI-assisted prose was reviewed and edited by the author for technical accuracy. The core algorithmic content (strapdown mechanization equations, UKF sigma-point formulation, and particle filter resampling strategies) is based directly on established references and was verified by the author independently of any AI-generated suggestions.

References

- 236
237 Brodovsky, J., & Dames, P. (2026a). Navigation in GNSS-denied environments using
238 MEMS-grade sensors and geophysical anomalies: A UKF approach. *Proceedings of*
239 *the 2026 International Technical Meeting of the Institute of Navigation*, 155–164.
240 <https://doi.org/https://doi.org/10.33012/2026.20508>
- 241 Brodovsky, J., & Dames, P. (2026b). Navigation in GNSS-denied environments using MEMS-
242 grade sensors and geophysical anomalies: A particle filter approach. *Proceedings of the*
243 *ION Pacific PNT Meeting*.
- 244 Dellaert, F. (2012). GTSAM: A factor graph based nonlinear estimation library. *Georgia*
245 *Institute of Technology*. <https://gtsam.org>
- 246 Fang, Y. (2019). *Gnss-ins-sim: GNSS/INS simulator for testing and prototyping navigation*
247 *algorithms*. <https://github.com/Aceinna/gnss-ins-sim>
- 248 Geneva, P., Eckenhoff, K., Lee, W., Yang, Y., & Huang, G. (2020). OpenVINS: A research
249 platform for visual-inertial estimation. *Proceedings of the IEEE International Conference*
250 *on Robotics and Automation (ICRA)*. https://github.com/rpng/open_vins
- 251 Groves, P. (n.d.). *Principles of GNSS, inertial, and multisensor integrated navigation systems*
252 (2nd ed.). Artech House. ISBN: 978-1-60807-005-3
- 253 National Geospatial-Intelligence Agency. (2014). *Department of defense world geodetic*
254 *system 1984: Its definition and relationships with local geodetic systems* (No.
255 NGA.STND.0036_1.0.0_WGS84). National Geospatial-Intelligence Agency.
- 256 NOAA NCEI Geomagnetic Modeling Team and British Geological Survey. (2024). *World*
257 *magnetic model 2025*. NOAA National Centers for Environmental Information. <https://doi.org/10.25921/aqfd-sd83>
- 258
259 Nordmoen, J., & Feuerstein, M. (2022). *Nav-types*. <https://github.com/nordmoen/nav-types>